

Rapport d'Audit de Sécurité et Plan de Remédiation

TiziFit

Projet : Application Web TiziFit (Flask / Python)

Type d'évaluation : Audit de Sécurité Web (SAST, DAST, Code Review)

Date : 24 Août 2026

Statut : Terminé — Failles critiques et moyennes remédiées

1. Synthèse Exécutive

L'audit de sécurité réalisé sur l'application TiziFit avait pour objectif d'évaluer la résistance de l'application contre les attaques courantes du OWASP Top 10, d'identifier les vulnérabilités dans le code source et d'assurer un niveau de sécurité optimal avant déploiement.

L'analyse a révélé 1 vulnérabilité critique (Remote Code Execution via le mode Debug Flask) ainsi que plusieurs failles de sévérité moyenne à faible (absence de protection CSRF et défauts de validation des entrées utilisateur). L'ensemble de ces failles a été corrigé avec succès.

Tableau récapitulatif des vulnérabilités

ID	Vulnérabilité	Sévérité	CWE	Initial	Final
VULN-01	Console Debug Flask Active (debug=True)	CRITIQUE	CWE-94 / CWE-95	Non Corrigé	CORRIGÉ
VULN-02	Absence de Tokens Anti-CSRF	MOYENNE	CWE-352	Non Corrigé	CORRIGÉ
VULN-03	Secrets & Configurations Codés en Dur	MOYENNE	CWE-798 / CWE-259	Non Corrigé	CORRIGÉ
VULN-04	Absence de Validation des Bornes (Input)	FAIBLE	CWE-20	Non Corrigé	CORRIGÉ
VULN-05	Contrôle d'Accès IDOR / BOLA	INFO	CWE-284 / CWE-639	SÉCURISÉ	CONFORME

2. Méthodologie d'Audit

L'évaluation a été menée selon une approche hybride en 4 phases successives :

1. Reconnaissance & Cartographie : Inventaire automatique des 12 routes/endpoints de l'application via un script d'analyse AST Python (audit_carto.py).
2. Analyse Statique du Code (SAST) : Exécution de l'outil d'analyse automatisé Bandit sur le fichier app.py.
3. Analyse Logique & Revue de Code Manuelle (DAST/Code Review) : Vérification de la gestion des sessions, des requêtes SQL (ORM SQLAlchemy) et du contrôle d'accès utilisateur (IDOR).
4. Remédiation & Hardening : Implémentation des correctifs et re-scan de validation.

3. Détail des Vulnérabilités et Remédiations

VULN-01 : Exécution de Code à Distance via Console Debug Werkzeug

Sévérité : CRITIQUE (High)

Description : L'application était configurée avec `app.run(debug=True)`. En cas d'erreur ou d'exception en production, Flask affiche la console interactive Werkzeug, permettant à un attaquant d'exécuter du code Python arbitraire sur le serveur hôte.

Remédiation :

- Désactivation du mode debug par défaut via la gestion dynamique des variables d'environnement (`FLASK_DEBUG=False`).

- Conditionnement du lancement du serveur dans app.py :

```
if __name__ == '__main__':
    debug_mode = os.getenv('FLASK_DEBUG', 'False').lower() in ['true', '1']
    app.run(debug=debug_mode)
```

VULN-02 : Absence de Protection CSRF (Cross-Site Request Forgery)

Sévérité : MOYENNE (Medium)

Description : Les routes acceptant des requêtes POST (/ajouter_calories, /maj_poids, /coaching) ne vérifiaient aucun token CSRF. Un attaquant pouvait forger un lien malveillant poussant un utilisateur connecté à modifier ses données à son insu.

Remédiation : Intégration de l'extension Flask-WTF (CSRFProtect) et activation de la protection CSRF globale :

```
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)
```

VULN-03 : Clés Secrètes et Paramètres Sensibles en Dur

Sévérité : MOYENNE (Medium)

Description : La valeur SECRET_KEY servant à signer les cookies de session Flask était définie directement dans le code source ou absente, ce qui exposait les sessions au risque de falsification si le code était publié.

Remédiation : Migration des configurations sensibles vers un fichier d'environnement .env exclu du versionnement Git (.gitignore) et chargement sécurisé avec python-dotenv :

```
import os
from dotenv import load_dotenv

load_dotenv()
app.config['SECRET_KEY'] = os.getenv('SECRET_KEY')
```

VULN-04 : Défaut de Validation des Bornes de Données Métier

Sévérité : FAIBLE (Low)

Description : La fonction maj_poids() convertissait l'entrée utilisateur en nombre flottant sans appliquer de bornes logiques. Un utilisateur pouvait soumettre des poids négatifs ou irréalistes (ex. -10 kg ou 99999 kg), corrompant le calcul automatique des besoins caloriques.

Remédiation : Ajout d'une vérification explicite des bornes (entre 30,0 kg et 300,0 kg) avant enregistrement en base de données :

```
if nouveau_poids is not None:
    if nouveau_poids < 30.0 or nouveau_poids > 300.0:
        flash("Veuillez entrer un poids valide compris entre 30 et 300 kg.", "danger")
        return redirect(url_for('track'))
```

4. Résultats de la Re-Validation

Après application des correctifs ci-dessus, une nouvelle analyse statique a été exécutée sur l'application :

```
python -m bandit -r app.py
```

Résultat de l'analyse :

```
Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 0
Test results:
  No issues identified.
```

L'application TiziFit présente désormais un niveau de sécurité robuste, conforme aux meilleures pratiques du développement web sécurisé (DevSecOps).

Conclusion : Les vulnérabilités identifiées lors de l'audit ont été remédiées et la re-validation automatisée n'a identifié aucun problème de sécurité dans le périmètre analysé.